

Snail Run

By Alex Ryström (060713-8799) and Astrid Leonard (021220-3103)
2025-10-10

Objective and Requirements

The objective of this project was to design and implement a simple endless runner game on the DE10-Lite board, using primarily C with small snippets of Assembly for interrupt handling. The game challenges the player, the character being a snail, to survive as long as possible by switching between three fixed lanes to avoid obstacles while accumulating points. The core requirements were that the game be rendered to a VGA display, allowed responsive input using the on-board switches, and displayed a numeric score on the 7-segment displays. While we initially aimed to implement multiple user profiles and a leaderboard, these features were not included in the final version. Additional features that were implemented included scaling difficulty, both through increased obstacle speed and frequency, and a reset function to replay the game using the button (key 1) on the board.

Solution

The game was implemented primarily in C, with small snippets of Assembly used for interrupt handling. The entire game was structured in a single file, with functions to handle graphics and game logic, and a main loop running the game.

The main loop operates by repeatedly calling functions to update the game state and then draw the corresponding frame, as well as updating the 7-segment display and swap screen buffers. This is done as long as the truth value *game_over* is false. Once a collision is detected, this is set to true, the main game loop exits and a “GAME OVER” message is displayed on the screen. From here the player can choose to reset the game by pressing the button on the board.

The *update_game_state()* function updates the state of the obstacle(s) on screen. It moves every obstacle along towards the player, then checks for collision with the player, which if true triggers *game over*. It also checks if an obstacle has reached the end of the screen, at which point the player's score is incremented, as well as the speed at which the obstacles are moving and the rate at which they spawn.

The *draw_game()* function fills the screen with a background colour, then draws the player and lane boundaries (alternating between two different frames to simulate moving animation), and finally draws all active obstacles.

Aside from these main functions the code includes a set of predefined values, global variables, structs, functions for game logic such as retrieving player lane and spawning obstacles, functions for drawing graphics such as drawing a pixel, filling screen, drawing sprites, and swapping buffers, and small helper functions such as random number generator, delay, and getting button and switch states.

Verification

Verification was performed at multiple levels. Game logic was first tested by implementing a functioning version on a PC using keyboard inputs before adapting it for the board. Individual functions were tested as they were developed, for example score output which was initially displayed on the screen to verify correctness before being implemented on the 7-segment displays (although we

ended up keeping the test code for screen output in the final version). Obstacle movement and spawning were also tested before graphics or collision checks were added. Finally, integrated system testing and stress tests under higher obstacle densities confirmed the stability and functionality of the complete game.

Contributions

Alex developed the main skeleton of the game, first as a PC version with keyboard input and later modified it to run on the board, writing most of the functions for the finalized game, including implementing interrupts for input handling via switches.

Astrid focused on the graphics; writing the initial code for displaying pixels on the monitor and buffer swapping, as well as designing and drawing the graphics. She also implemented the code for spawning multiple objects, tracking and displaying score on the 7-segment displays, and collision detection.

Both of us collaborated on modifying and adding code throughout the project. We worked mostly in parallel, while still discussing most features and at times working on the same code, making it difficult to precisely separate responsibilities. In general Alex focused more on game logic and Astrid more on graphics, but we were both involved in both parts of development.

Reflections

Overall, we feel that this project provided valuable experience in low-level programming and hardware-software integration. Working with the board particularly deepened our understanding of how graphics rendering and input handling operate at the hardware level. We encountered issues with screen flickering but eventually resolved them by correctly implementing buffer swapping. It was rewarding to see the game come to life, especially when adding detailed graphics and small animations.